



**UNSW**  
SYDNEY

# COMP4141

Theory of Computation

Lecture 16: Oracles and The Polynomial Time Hierarchy

# Administrivia

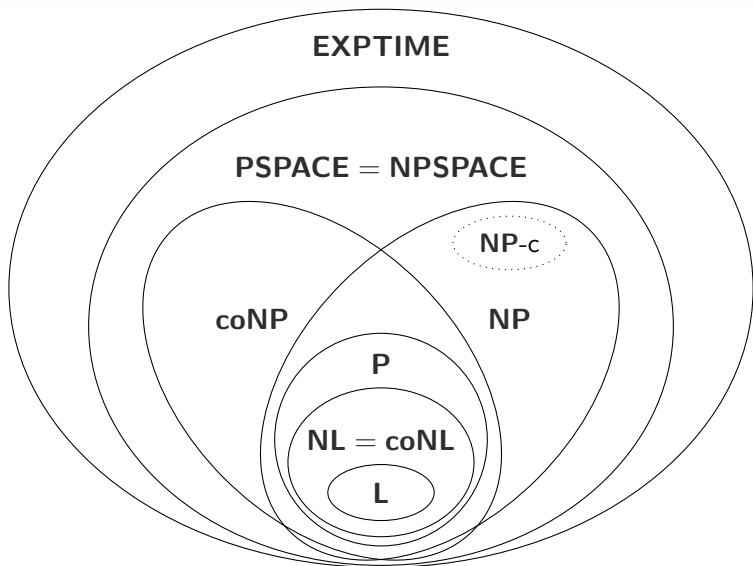
- Final exam: 3 May (24 hours)
- Formatif tasks: 6 available, do at least 3 before final exam
- Formatif feedback asap
- Tutorial solutions up asap
- Recap lecture next week (Wednesday)
- Consultations continue until final exam
- myExperience due soon
- Pizza tomorrow!

## More TCS...

If you like the topics covered in this course:

- COMP3153/9153 Algorithmic Verification
- COMP6741 Algorithms for intractable problems
- COMP3151/9154 Foundations of Concurrency
- Advanced topics in TCS reading group: send me an email expressing interest

# Complexity classes



## Complexity summary

| Class          | Characterization                                  | Problem      | Notes            |
|----------------|---------------------------------------------------|--------------|------------------|
| <b>P</b>       | $\bigcup_{k \in \mathbb{N}} \text{TIME}(n^k)$     | HORN-SAT     |                  |
| <b>NP</b>      | $\bigcup_{k \in \mathbb{N}} \text{NTIME}(n^k)$    | SAT, 3SAT    | Poly. verifiable |
| <b>PSPACE</b>  | $\bigcup_{k \in \mathbb{N}} \text{SPACE}(n^k)$    | QBF          | = <b>NPSPACE</b> |
| <b>coNP</b>    | Languages $L$ such that $\bar{L} \in \text{NP}$   | VALIDITY     | Poly. refutable  |
| <b>EXPTIME</b> | $\bigcup_{k \in \mathbb{N}} \text{TIME}(2^{n^k})$ | BOUNDED-HALT | $\neq \text{P}$  |
| <b>L</b>       | <b>SPACE</b> (log $n$ )                           | UND. PATH    |                  |
| <b>NL</b>      | <b>NSPACE</b> (log $n$ )                          | PATH, 2SAT   | = <b>coNL</b>    |

# Outline

Relativization

Alternation

Polynomial Time Hierarchy

# Relativization

Recall Lecture 8:

## Definition

An **oracle** for a language  $B$  is an external device that can determine whether a given word  $w$  is a member of  $B$ . An **oracle (for  $B$ ) Turing Machine**,  $M^B$ , is a Turing machine that has the additional capacity of querying an oracle for  $B$ .

If an oracle TM  $M^B$  decides  $A$  then we say that  $A$  is *decidable relative to  $B$* .

Language  $A$  is *Turing reducible* to language  $B$  (or  $A \leq_T B$ ) if  $A$  is decidable relative to  $B$ .

# Oracle complexity classes

## Definition

For complexity classes  $\mathcal{A}, \mathcal{B}$  let  $\mathcal{A}^{\mathcal{B}} = \bigcup_{L \in \mathcal{B}} \mathcal{A}^L$  be the class of languages decided by an  $\mathcal{A}$ -computation using an oracle in  $\mathcal{B}$ .

## NB

*We assume each call to an oracle takes  $O(1)$  time.*

## Example

$\mathbf{P}^O$  is the class of languages decided by a polynomial-time oracle TM using oracle  $O$ . (Similarly for  $\mathbf{NP}^O$ .)

$\mathbf{NP} \subseteq \mathbf{P}^{\text{SAT}}$  and  $\mathbf{coNP} \subseteq \mathbf{P}^{\text{SAT}}$ .



# Oracle example

## Example

Consider the decision problem:

MIN-F

**Input:** A Boolean formula  $\phi$

**Question:** Is  $\phi$  minimal?

where a Boolean formula  $\phi$  is *minimal* if there does not exist a shorter formula  $\psi$  such that  $\phi \equiv \psi$  (equivalently,  $\phi \leftrightarrow \psi$  is valid).

# Oracle example

## Example

Consider the decision problem:

MIN-F

**Input:** A Boolean formula  $\phi$

**Question:** Is  $\phi$  minimal?

where a Boolean formula  $\phi$  is *minimal* if there does not exist a shorter formula  $\psi$  such that  $\phi \equiv \psi$  (equivalently,  $\phi \leftrightarrow \psi$  is valid).

It is not known whether  $\text{MIN-F} \in \mathbf{coNP}$  (or even  $\mathbf{NP}$ ) .

# Oracle example

## Example

Consider the decision problem:

MIN-F

**Input:** A Boolean formula  $\phi$

**Question:** Is  $\phi$  minimal?

But  $\overline{\text{MIN-F}} \in \mathbf{NP}^{\text{SAT}}$  as witnessed by the oracle NTM:

On input  $\langle \phi \rangle$ :

- 1 Non-deterministically guess a smaller formula  $\psi$ .
- 2 Ask the oracle whether  $\langle \neg(\phi \leftrightarrow \psi) \rangle \in \text{SAT}$  and if it accepts, *REJECT*; otherwise *ACCEPT*.

# Relativization and Diagonalization

Any theorem proved about TMs by using only methods based on

- I string representations of TMs
- II simulation of one TM by another without much overhead in time/space

lifts to oracle machines.

## Examples

- Undecidability of  $A_{TM}$ :
  - $\{\langle M, x \rangle : M \text{ is an oracle TM for } B \text{ that accepts } x\}$  cannot be decided by an oracle TM for  $B$
- Time/Space hierarchy theorems

# $P \stackrel{?}{=} NP$ and Diagonalization

That the resolution of  $P \stackrel{?}{=} NP$  can not be such a theorem follows from:

## Theorem (Baker, Gill, Solovay 1975)

*Oracles  $A$  and  $B$  exist whereby  $P^A = NP^A$  and  $P^B \neq NP^B$ .*

# Proof of $\exists A (\mathbf{P}^A = \mathbf{NP}^A)$

$A$  could be QBF:

|                                                       |                                       |
|-------------------------------------------------------|---------------------------------------|
| $\mathbf{NP}^{\text{QBF}} \subseteq \mathbf{NPSPACE}$ | by $\text{QBF} \in \mathbf{PSPACE}$   |
| $= \mathbf{PSPACE}$                                   | by Savitch's theorem                  |
| $\subseteq \mathbf{P}^{\text{QBF}}$                   | QBF is $\mathbf{PSPACE}$ -complete    |
| $\subseteq \mathbf{NP}^{\text{QBF}}$                  | by $\mathbf{P} \subseteq \mathbf{NP}$ |



## Proof of $\exists B (\mathbf{P}^B \neq \mathbf{NP}^B)$

Iteratively construct a  $B$  (and its complement  $B'$ ) such that in the end  $U_B \in \mathbf{NP}^B \setminus \mathbf{P}^B$  where

$$U_B = \{ 1^n \mid \Sigma^n \cap B \neq \emptyset \} .$$

That  $U_B \in \mathbf{NP}^B$  is easy:

“On input  $1^n$  guess  $x \in \Sigma^n$  and *accept* iff the oracle confirms  $x \in B$ .”

## Proof of $\exists B (\mathbf{P}^B \neq \mathbf{NP}^B)$ cont.

Initially,  $B = B' = \emptyset$ . For stage  $i$  of the construction, let  $M_i^?$  be the  $i$ 'th polynomial-time oracle TM running in w.l.o.g. in time  $n^i$ .

Let  $m$  exceed the length of all strings in  $B \cup B'$  so far, and also  $m^i < 2^m$ .

We'll ensure that  $U_B$  and  $M_i^B$  disagree on  $1^m$ .

- 1 Simulate  $M_i^?$  on  $1^m$  by answering queries  $x$  to the oracle with "yes" if  $x \in B$ , "no" if  $x \in B'$ , otherwise we also answer "no" and add  $x$  to  $B'$ .
- 2 If  $M_i^?$  accepts  $1^m$  then we put all strings of length  $m$  into  $B'$ ; otherwise, we add the first string of length  $m$  neither in  $B$  nor in  $B'$  to  $B$ . Such a string exists because  $M_i^?$  can have queried at most  $m^i < 2^m$  strings of length  $m$  and none were queried ever before.

It follows that no  $M_i^B$  will decide  $U_B$  and thus  $U_B \notin \mathbf{P}^B$ . □



# Outline

Relativization

**Alternation**

Polynomial Time Hierarchy

# Motivation

## Recall

- *nondeterministic* TM's, accept if some branch of the computation tree accepts (used for **NP**)
- *co-nondeterministic* TM's, accept if all branches of the computation tree accepts (used for **coNP**)

**Alternating** Turing machines combine the two acceptance modes into one type of machine .....

# Alternating Turing Machines

## Definition

An *alternating Turing Machine (ATM)*

$$M = (Q, \ell, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$$

consists of:

- $Q$ , a finite set of states
- $\ell : Q \rightarrow \{\forall, \exists\}$ , a labelling of states as *universal* or *existential*
- $\Sigma$ , the input symbol alphabet,  $\sqcup \notin \Sigma$
- $\Gamma \supseteq \Sigma$ , the tape symbol alphabet,  $\sqcup \in \Gamma$
- $\delta : Q \times \Gamma \rightarrow 2^{Q \times \Gamma \times \{L, R\}}$ , the transition function
- $q_0 \in Q$ , the start state
- $q_{\text{accept}} \in Q$ , the accept state
- $q_{\text{reject}} \in Q$ , the reject state

# Language of an ATM

An ATM  $M$  runs on a word  $w$  as if it were an NTM, creating a tree  $T_M(w)$  of TM configurations (or a directed graph if we identify nodes with identical configurations).

## Definition (ATM acceptance)

Mark nodes of  $T_M(w)$ , proceeding from leaves to the root, as follows:

- 1 every (accepting) configuration  $xq_{\text{accept}}y$  is marked,
- 2  $c$  is marked if  $\ell(c) = \exists$  and  $c$  has *some* marked successor in  $T_M(w)$ , and
- 3  $c$  is marked if  $\ell(c) = \forall$  and  $c$  *all* successors in  $T_M(w)$  are marked .

If the root of  $T_M(w)$  (the initial configuration) is marked at the end of this process, then  $M$  accepts  $w$ .

# ATIME and ASPACE

Definitions of time and space complexity need not be changed.

## Definition

Let  $t : \mathbb{N} \rightarrow \mathbb{N}$ .

The *alternating time complexity class*, **ATIME**( $t(n)$ ) is the collection of all languages that are decidable by an  $O(t(n))$  time ATM.

The *alternating space complexity class*, **ASPACE**( $t(n)$ ) is the collection of all languages that are decidable by an  $O(t(n))$  space ATM.

$$\mathbf{AP} = \bigcup_{k \in \mathbb{N}} \mathbf{ATIME}(n^k)$$

$$\mathbf{APSPACE} = \bigcup_{k \in \mathbb{N}} \mathbf{ASPACE}(n^k)$$

$$\mathbf{AL} = \mathbf{ASPACE}(\log n)$$

# Alternation example

## Example

Recall that it is not known whether

$$\text{MIN-F} = \{ \langle \phi \rangle \mid \phi \text{ is minimal} \}$$

is in **NP** or **coNP**. All we know so far is that  $\overline{\text{MIN-F}} \in \mathbf{NP}^{\text{SAT}}$ .

# Alternation example

## Example

Recall that it is not known whether

$$\text{MIN-F} = \{ \langle \phi \rangle \mid \phi \text{ is minimal} \}$$

is in **NP** or **coNP**. All we know so far is that  $\overline{\text{MIN-F}} \in \text{NP}^{\text{SAT}}$ .

Consider the following alternating algorithm:

On input  $\langle \phi \rangle$ :

- 1 Universally select a shorter formula  $\psi$ .
- 2 Existentially select an interpretation  $\pi$  of  $\phi$
- 3 Evaluate  $\phi$  and  $\psi$  on  $\pi$
- 4 *Accept* if the results differ and *reject* otherwise.

# Alternation example

## Example

Recall that it is not known whether

$$\text{MIN-F} = \{ \langle \phi \rangle \mid \phi \text{ is minimal} \}$$

is in **NP** or **coNP**. All we know so far is that  $\overline{\text{MIN-F}} \in \mathbf{NP}^{\text{SAT}}$ .

Consider the following alternating algorithm:

On input  $\langle \phi \rangle$ :

- ❶ Universally select a shorter formula  $\psi$ .
- ❷ Existentially select an interpretation  $\pi$  of  $\phi$
- ❸ Evaluate  $\phi$  and  $\psi$  on  $\pi$
- ❹ *Accept* if the results differ and *reject* otherwise.

This shows that  $\text{MIN-F} \in \mathbf{AP}$ .



# Time, Space, and Alternation

## Theorem

- 1  $\mathbf{ATIME}(t(n)) \stackrel{(a)}{\subseteq} \mathbf{SPACE}(t(n)) \stackrel{(b)}{\subseteq} \mathbf{ATIME}(t(n)^2)$  if  $t(n) \geq n$ .
- 2  $\mathbf{ASPACE}(t(n)) = \mathbf{TIME}(2^{O(t(n))})$  if  $t(n) \geq \log n$ .

## Corollary

- $\mathbf{AL} = \mathbf{P}$ ,
- $\mathbf{AP} = \mathbf{PSPACE}$ , and
- $\mathbf{APSPACE} = \mathbf{EXPTIME}$ .

# Proof Ideas

## Proof of 1.

(a) **ATIME**( $t(n)$ )  $\subseteq$  **SPACE**( $t(n)$ ):

Simulate the ATM, do DFS to do the marking.

This gives **SPACE**( $t(n)^2$ ), one  $t(n)$  for the recursion depth and one  $t(n)$  for the configuration size.

Reduce the latter to constant size by merely recording the choices made at each non-deterministic step, and recomputing the configuration from the start when backtracking.

# Proof Ideas

## Proof of 1.

(b) **SPACE**( $t(n)$ )  $\subseteq$  **ATIME**( $t^2(n)$ ):

As in Savitch's theorem, the ATM uses binary search to determine whether the simulated TM could reach the accepting configuration in  $2^{dt(n)}$  steps.

Rather than iterating over all possible midpoint configurations  $c_m$ , the ATM can use one big existential guess of  $t(n)$  steps to construct  $c_m$ , (and then universally branch into two recursive calls).

# Proof Ideas

## Proof of 2.

("⊆") **ASPACE**( $t(n)$ ) ⊆ **TIME**( $2^{O(t(n))}$ ):

Similar to the proof of **SPACE**( $t(n)$ ) ⊆ **TIME**( $2^{O(t(n))}$ ), on input  $w$ , construct the directed acyclic computation graph containing the configurations of the simulated ATM; first mark the accepting nodes and then accept the word according to the definition of ATM acceptance.

# Proof Ideas

## Proof of 2.

(“ $\supseteq$ ”) **ASPACE**( $t(n)$ )  $\supseteq$  **TIME**( $2^{O(t(n))}$ ):

Simulate a deterministic  $2^{f(n)}$  machine  $M$  by an  $O(f(n))$  space ATM  $S$ , where  $f(n) = O(t(n))$ . W.l.o.g.  $M$  accepts after erasing all tape content and moving all the way to the left.

On input  $w$ ,  $M$  goes through a sequence of configurations. We can't even store a single one of those in  $S$  because it could be too long!

# Proof Ideas

## Proof of 2.

("⊇") **ASPACE**( $t(n)$ )  $\supseteq$  **TIME**( $2^{O(t(n))}$ ):

Let's encode the sequence of configurations as a  $2^{f(|w|)} \times 2^{f(|w|)}$  grid where a cell contains either a tape symbol or a tape symbol and a state if that's where the head of  $M$  is.

$S$  then uses a recursive procedure  $R(i, j, d)$  where  $i, j$  are pointers of size  $\lceil f(|w|) \rceil$  in binary to a cell and  $d$  is a cell contents, to check the bottom left grid cell  $i = 2^{f(|w|)}$ ,  $j = 1$  has contents  $d = (q_{\text{accept}}, \sqcup)$ .

# Proof Ideas

## Proof of 2.

$R(i, j, d)$

- ① if  $i = 1$  then *accept* if  $d$  is consistent with the  $j$ 'th cell of the initial configuration  $q_0w$  in this representation; else *reject*.
- ②  $\exists$ -guess the contents  $a, b, c$  of the parent cells  $[i - 1, j - 1]$ ,  $[i - 1, j]$ ,  $[i - 1, j + 1]$ 
  - ① if the parent cells with contents  $a, b, c$  shouldn't have the child cell  $[i, j]$  with contents  $d$ , *reject*
  - ②  $\forall$ -recurse into  $R(i - 1, j - 1, a)$ ,  $R(i - 1, j, b)$ ,  $R(i - 1, j + 1, c)$ .

Even though the recursion depth is  $2^{f(|w|)}$ , the ATM can do with  $O(f(w))$  space because it need not store any of the arguments of previous recursive calls:  $R$  only ever returns *accept* or *reject*.



## Alternative Proof of $\mathbf{PSPACE} \subseteq \mathbf{AP}$

We show  $\mathbf{QBF} \in \mathbf{AP}$  by giving a polynomial time ATM.

On input  $Q_1x_1 \dots Q_kx_k\phi(x_1, \dots, x_k)$  where the  $Q_i \in \{\exists, \forall\}$ :

- 1 for  $1 \leq i \leq k$ 
  - $Q_i$ -guess a value  $v_i \in \{\text{FALSE}, \text{TRUE}\}$  for  $x_i$
- 2 evaluate  $\phi(v_1, \dots, v_k)$  and *accept* if it's true; *reject* otherwise.



# Outline

Relativization

Alternation

Polynomial Time Hierarchy

# Polynomial Time Hierarchy

## Definition

A  $\Sigma_i$ -ATM is an ATM whose runs begin at an  $\exists$  state, and alternate, i.e., switch the  $\{\exists, \forall\}$ -type of state, at most  $i - 1$  times.

A  $\Pi_i$ -ATM is an ATM whose runs begin at a  $\forall$  state, and alternate at most  $i - 1$  times.

# Polynomial Time Hierarchy

## Definition

A  $\Sigma_i$ -ATM is an ATM whose runs begin at an  $\exists$  state, and alternate, i.e., switch the  $\{\exists, \forall\}$ -type of state, at most  $i - 1$  times.

A  $\Pi_i$ -ATM is an ATM whose runs begin at a  $\forall$  state, and alternate at most  $i - 1$  times.

## Definition

$\Sigma_i \mathbf{TIME}(f(n))$  is the class of languages  $\Sigma_i$ -ATMs can decide in  $O(f(n))$  time.

$$\Sigma_i \mathbf{P} = \bigcup_k \Sigma_i \mathbf{TIME}(n^k).$$

Similarly, we define  $\Pi_i \mathbf{TIME}(f(n))$ , and  $\Pi_i \mathbf{P}$ .

## NB

$$\mathbf{NP} = \Sigma_1 \mathbf{P} \text{ and } \mathbf{coNP} = \Pi_1 \mathbf{P}.$$

# Polynomial Time Hierarchy

## Definition

A  $\Sigma_i$ -ATM is an ATM whose runs begin at an  $\exists$  state, and alternate, i.e., switch the  $\{\exists, \forall\}$ -type of state, at most  $i - 1$  times.

A  $\Pi_i$ -ATM is an ATM whose runs begin at a  $\forall$  state, and alternate at most  $i - 1$  times.

## Definition

$\Sigma_i \mathbf{TIME}(f(n))$  is the class of languages  $\Sigma_i$ -ATMs can decide in  $O(f(n))$  time.

$$\Sigma_i \mathbf{P} = \bigcup_k \Sigma_i \mathbf{TIME}(n^k).$$

Similarly, we define  $\Pi_i \mathbf{TIME}(f(n))$ , and  $\Pi_i \mathbf{P}$ .

## Definition

$\mathbf{PH} = \bigcup_i \Sigma_i \mathbf{P} = \bigcup_i \Pi_i \mathbf{P}$  is called the *polynomial time hierarchy*.

# Examples

## Example

The **AP** algorithm for MIN-F:

On input  $\langle \phi \rangle$ :

- 1 Universally select a shorter formula  $\psi$ .
- 2 Existentially select an interpretation  $\pi$  of  $\phi$
- 3 Evaluate  $\phi$  and  $\psi$  on  $\pi$
- 4 *Accept* if the results differ and *reject* otherwise.

Shows that  $\text{MIN-F} \in \Pi_2\text{P}$ .

# Examples

## Example

Other examples include:

- Games with a fixed number of moves
- QBF with a fixed number of  $\exists$ - $\forall$  alternations

# Polynomial Time Hierarchy using Oracles

Alternatively, we could have defined

## Definitions

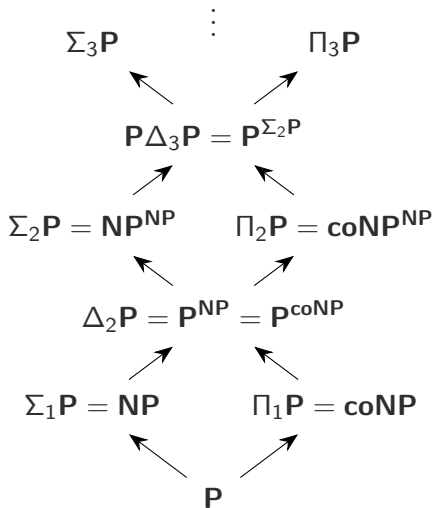
$\Delta_0\mathbf{P} = \Sigma_0\mathbf{P} = \Pi_0\mathbf{P} = \mathbf{P}$ ; and for  $i \geq 0$

$$\Delta_{i+1}\mathbf{P} = \mathbf{P}^{\Sigma_i\mathbf{P}}$$

$$\Sigma_{i+1}\mathbf{P} = \mathbf{NP}^{\Sigma_i\mathbf{P}}$$

$$\Pi_{i+1}\mathbf{P} = \mathbf{coNP}^{\Sigma_i\mathbf{P}}$$

# PSPACE





# Open Questions

## Open questions

- Does **PH = PSPACE**?
- Is the polynomial time hierarchy strict?